

PIGGY ME

Documento de Arquitetura

Versão [1.2]

Tabela - Integrantes do Grupo

Mat.	Nome	Função (responsabilidade)	Pontos de participação na elaboração
242032237	Cauã Mendes Coelho	desenvolvedor	10
242004546	Calynne Ellen da Silva Jesus	proposito e escopo	10
242004715	Giovana Rocha e Silva	Metas e restrições arquiteturais; restrições adicionais.	10
222021862	Daniel Paz Dos Passos	Visão Estrutural	9 ALUNO TRANCOU
	Isaias Moraes Pacheco	2.1 Definições	8
242015906	Luis Gabriel dos Santos Galeno	2.6 visão de implantação	7
242015951	Rodrigo Dutra Ferreira	desenvolvedor	10
242015719	Thiago Kauã de Sousa Cardoso	desenvolvedor	6 ALUNO TRANCOU
211063004	Rafael Magalhães Merenciano	2.5.2 Visão de organização Lógica	10
242015399	Rafael Araujo Queiroz	2.3 - Detalhamento	10
242015352	Vítor da Costa Rossi de Oliveira	2.5.1 visão uso	10

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
14/04	1.1	revisão de tabelas	Calynne Ellen da Silva Jesus
02/06	1.2	revisão/atualização da metade do documento	Calynne Ellen da Silva Jesus
03/06	1.3	revisão/atualização da metade do documento	Vítor da Costa Rossi De Oliveira

Sumário

1	Introdução.....	6
1.1	Propósito.....	6
1.2	Escopo.....	6
2	Representação Arquitetural.....	6
2.1	Definições.....	6
2.2	Justifique sua escolha.....	6
2.3	Detalhamento.....	7
2.4	Metas e restrições arquiteturais.....	9
2.5	Visões.....	10
2.5.1	Visão uso.....	10
2.5.2	Visão de organização lógica.....	13
2.5.3	Visão estrutural.....	15
2.6	Visão de Implantação.....	17
2.7	Restrições adicionais.....	18
3	Bibliografia.....	20

1 Introdução

1.1 Propósito

Este documento descreve a arquitetura do sistema sendo desenvolvido pelo grupo, na disciplina de MDS – Métodos de Desenvolvimento de Software – edição do primeiro semestre de 2026, para o sistema PiggyMe, a fim de fornecer uma visão abrangente do sistema para desenvolvedores, testadores e demais interessados em aspectos relacionados às tecnologias a serem usadas no desenvolvimento.

1.2 Escopo -

O detalhamento do escopo se encontra no documento “Visão do Produto e do Projeto – PiggyMe” que contém a declaração de escopo do projeto e produto. Porém, em linhas gerais o escopo do produto compreende o desenvolvimento de uma plataforma web de gestão financeira pessoal, permitindo aos usuários registrar, categorizar e acompanhar receitas e despesas, gerar relatórios financeiros, e utilizar recursos de gamificação para incentivo ao letramento financeiro.

2 Representação Arquitetural

2.1 Definições

O sistema PiggyMe seguirá uma arquitetura baseada no padrão MVC (Model–View–Controller), combinada com uma arquitetura em camadas no front-end. Essa decisão arquitetural foi adotada pela equipe devido à necessidade de separação clara entre interface, regras de negócio e manipulação de dados, facilitando a manutenção, organização e evolução do sistema ao longo das sprints.

No modelo MVC, o sistema é dividido em três componentes principais:

- **Model (Modelo):** responsável pela manipulação dos dados, regras de negócio e comunicação com o banco de dados;
- **View (Visão):** responsável pela interface gráfica e interação com o usuário;
- **Controller (Controlador):** responsável pelo processamento das requisições, validações e comunicação entre View e Model.

Além disso, o front-end será organizado em camadas de apresentação, lógica de interface e comunicação com API, promovendo modularização e reutilização de componentes.

Essa arquitetura será utilizada juntamente com uma API responsável pela comunicação entre o front-end e o back-end através de requisições HTTP, permitindo independência entre as camadas do sistema.

2.2 Justifique sua escolha.

A escolha da arquitetura MVC em conjunto com arquitetura em camadas foi realizada considerando os requisitos funcionais e não funcionais definidos nos documentos “Visão do Produto e do Projeto – PiggyMe” e “Declaração de Escopo do Projeto”.

De acordo com o documento de visão do produto, o PiggyMe consiste em uma plataforma web de gestão financeira pessoal voltada ao registro, categorização e acompanhamento de receitas e despesas, além de funcionalidades de gamificação e relatórios financeiros. Dessa

forma, tornou-se necessário adotar uma arquitetura que possibilitasse a separação adequada das responsabilidades do sistema e facilidade de manutenção futura.

A utilização do padrão MVC atende diretamente às necessidades do projeto, pois permite organizar o software em componentes independentes, facilitando o desenvolvimento incremental proposto pela metodologia Scrum utilizada pela equipe. Como o sistema possui funcionalidades distintas — autenticação, gerenciamento financeiro, relatórios e gamificação — a divisão em camadas reduz acoplamento entre módulos e melhora a reutilização de código.

Outro fator relevante para essa escolha foi a definição tecnológica do projeto. Conforme especificado no documento de visão do produto, o sistema utilizará JavaScript no front-end e back-end, além de banco de dados MySQL. O padrão MVC possui ampla compatibilidade com esse conjunto tecnológico, sendo amplamente utilizado em aplicações web que necessitam manipular dados de usuários, autenticação e comunicação constante com banco de dados.

A arquitetura escolhida também atende aos requisitos não funcionais estabelecidos no backlog do produto, principalmente:

- **Agilidade e desempenho:** a separação das responsabilidades facilita otimizações no carregamento das páginas e processamento das requisições;
- **Manutenibilidade:** a modularização do sistema facilita correções, testes e implementação de novas funcionalidades;
- **Segurança:** a existência de controllers e middlewares facilita a implementação de autenticação utilizando tokens JWT;
- **Responsividade e disponibilidade:** a separação entre front-end e back-end permite maior flexibilidade para adaptação da interface em diferentes dispositivos;
- **Escalabilidade futura:** a arquitetura possibilita expansão do sistema sem necessidade de reestruturação completa do software.

Além disso, a escolha por uma arquitetura monolítica baseada em MVC mostrou-se mais adequada ao escopo acadêmico do projeto, ao tamanho da equipe e ao cronograma definido nas sprints. Uma arquitetura de microsserviços, por exemplo, aumentaria significativamente a complexidade do desenvolvimento, implantação e comunicação entre módulos, sem trazer benefícios proporcionais ao contexto atual do PiggyMe.

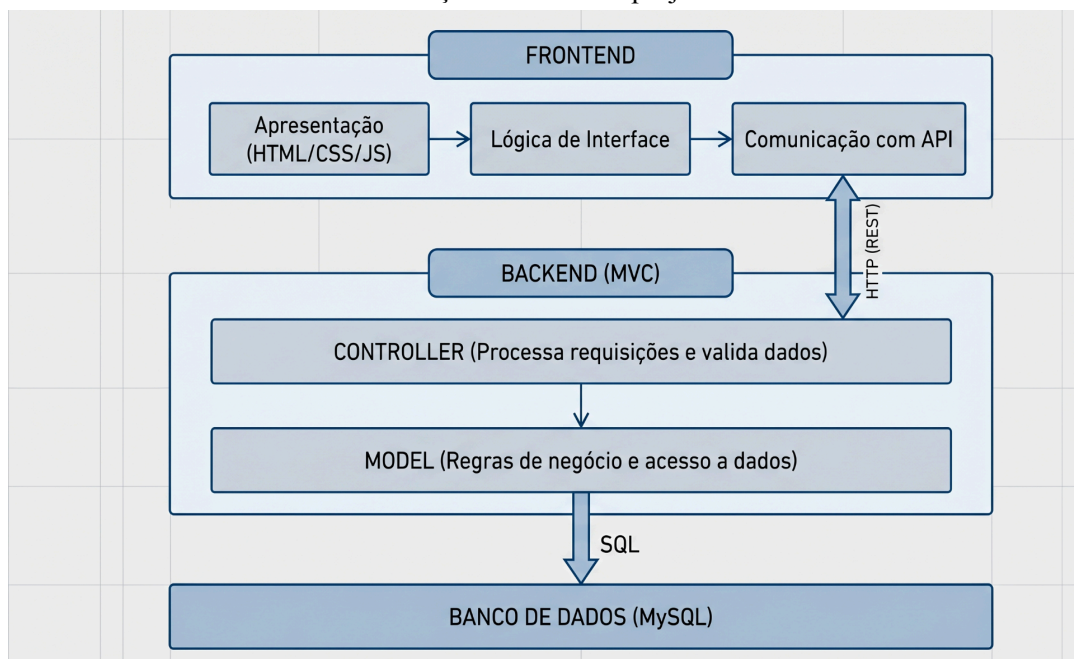
Portanto, a arquitetura adotada busca equilibrar simplicidade, organização, desempenho e facilidade de evolução, atendendo adequadamente aos objetivos funcionais e técnicos definidos pela equipe para o desenvolvimento do sistema.

2.3 Detalhamento

A estrutura do software **PiggyMe** fundamenta-se no padrão **MVC (Model-View-Controller)** integrado a uma **organização em camadas no front-end**, conforme ilustra a representação visual a seguir:

Figura 1 – Estrutura arquitetural do PiggyMe (MVC e Camadas)

base da arquitetura do PiggyMe é organizada para separar as responsabilidades do sistema entre diferentes camadas. A estrutura detalhada, ilustrada na Figura 1, demonstra a integração do padrão MVC no backend com a organização em camadas no frontend, o que facilita a manutenibilidade e a evolução contínua do projeto.



(fonte: autoria própria 2026)

Instanciação dos Elementos Arquiteturais no PiggyMe

Frontend — Arquitetura em Camadas

Camada de Apresentação Implementada com HTML, CSS e JavaScript, é responsável por renderizar todas as interfaces com as quais o usuário interage diretamente. No PiggyMe, engloba as telas de cadastro, login, controle de gastos, controle de receita, relatórios, gamificação, histórico de transações e conversão de moeda. Deve ser responsiva, garantindo acesso adequado tanto em computadores quanto em dispositivos móveis.

Camada de Lógica de Interface Responsável por validações no lado do cliente, como verificar se campos obrigatórios foram preenchidos, se o valor de uma transação é positivo ou se o formato do e-mail está correto, antes de enviar qualquer dado ao backend. Essa camada melhora a experiência do usuário ao fornecer feedback imediato sem necessidade de uma requisição ao servidor.

Camada de Comunicação com API Responsável por realizar as requisições HTTP entre o frontend e o backend. Envia e recebe dados em formato JSON, gerencia os tokens JWT de autenticação e trata os códigos de resposta retornados pelo servidor, como 200, 201, 400, 401 e 409

Backend — Padrão MVC

View (Visão) No contexto da arquitetura adotada, a View do backend é responsável por formatar e retornar as respostas das requisições ao frontend, estruturando os dados em JSON para consumo pela

camada de comunicação com API. Não possui responsabilidade sobre a renderização visual, que é delegada ao frontend.

Controller (Controlador) É o ponto de entrada das requisições HTTP no backend. Recebe as solicitações vindas do frontend, realiza validações de entrada, aciona os middlewares de segurança — como a verificação do token JWT — e delega as operações à camada Model. No PiggyMe, existem controllers específicos para cada funcionalidade do sistema, como autenticação, transações, relatórios, gamificação e conversão de moeda. Após o processamento pelo Model, o Controller retorna a resposta adequada à View.

Entradas esperadas: requisições HTTP com dados em JSON e token de autenticação no cabeçalho. Saídas produzidas: respostas HTTP com código de status e dados em JSON. Requisito de uso: o token JWT deve estar presente e válido em todas as rotas protegidas.

Model (Modelo) É a camada responsável pelas regras de negócio e pela comunicação com o banco de dados MySQL. No PiggyMe, o Model concentra as seguintes responsabilidades:

- **Usuário:** salvar, buscar por ID, buscar por e-mail, listar, atualizar e deletar usuários.
- **Criptografia:** criptografar e descriptografar senhas e dados sensíveis via tokens e cookies.
- **Transações:** salvar, buscar por usuário, buscar por período, buscar por categoria, somar despesas e deletar registros.
- **Relatório:** buscar dados para geração de gráficos, histórico completo e resumo mensal.
- **Gamificação:** salvar e atualizar pontuação XP, buscar ranking e atualizar o estado de saúde do personagem porquinho.
- **Conversão de moeda:** realizar o cálculo de conversão e buscar a taxa de câmbio.
- **Segurança:** gerar, validar, renovar e invalidar tokens JWT.
- **Autorização:** verificar permissões e identificar se o usuário é administrador.
- **Notificação:** enviar e-mails de recuperação de senha.

Banco de Dados — MySQL

Camada de persistência responsável por armazenar de forma íntegra e segura todos os dados do sistema, incluindo informações de usuários, transações financeiras, categorias, metas e pontuações de gamificação. A comunicação com essa camada é feita exclusivamente pelo Model, garantindo que nenhuma outra camada acesse diretamente o banco de dados, preservando o encapsulamento e a segurança da arquitetura.

Interfaces e Conectores

A comunicação entre o frontend e o backend ocorre exclusivamente por meio de uma **API REST**, utilizando requisições HTTP com troca de dados em formato JSON. Esse conector garante independência entre as camadas, permitindo que o frontend e o backend evoluam separadamente sem impacto direto um sobre o outro. A autenticação nas rotas protegidas é realizada por meio de **tokens JWT**, transmitidos no cabeçalho das requisições, assegurando que apenas usuários autenticados acessem dados sensíveis do sistema.

2.4 Metas e restrições arquiteturais

O sistema PiggyMe foi projetado considerando metas de desempenho, acessibilidade, organização do código e facilidade de manutenção, visando garantir uma experiência eficiente e intuitiva aos usuários.

Metas Arquiteturais:

- O sistema deve apresentar tempo de resposta de até 1 segundo na maioria das operações principais, como login, registro e visualização de gastos;
- A plataforma deve possuir interface responsiva, permitindo acesso adequado em computadores e dispositivos móveis;
- O sistema deve garantir facilidade de uso e acessibilidade, utilizando interfaces intuitivas e elementos visuais simples;
- O software deve possuir arquitetura modular, facilitando manutenção, testes e implementação de novas funcionalidades futuramente;
- O sistema deve incentivar o uso contínuo através dos recursos de gamificação e feedback visual ao usuário.

Restrições Arquiteturais:

- O desenvolvimento será realizado utilizando JavaScript no front-end e back-end. Assim como HTML e CSS no front-end para garantir uma interface única e adaptável para acesso em navegadores desktop e dispositivos móveis;
- O banco de dados adotado será o MySQL para armazenamento das informações financeiras e autenticação dos usuários;
- O projeto seguirá um desenvolvimento incremental utilizando SCRUM e práticas de XP;
- O sistema deverá funcionar diretamente em navegadores web modernos, sem necessidade de instalação adicional.

Padrões e diretrizes:

- Utilização de padrão de nomenclatura consistente para variações, funções e componentes;
- Organização do código em módulos e separação entre front-end, back-end e banco de dados;
- Comentários serão utilizados apenas em trechos necessários para facilitar a manutenção e auxiliar na comunicação;
- Uso do GitHub para controle das alterações do projeto.

2.5 Visões

2.5.1 Visão uso (V.R.)

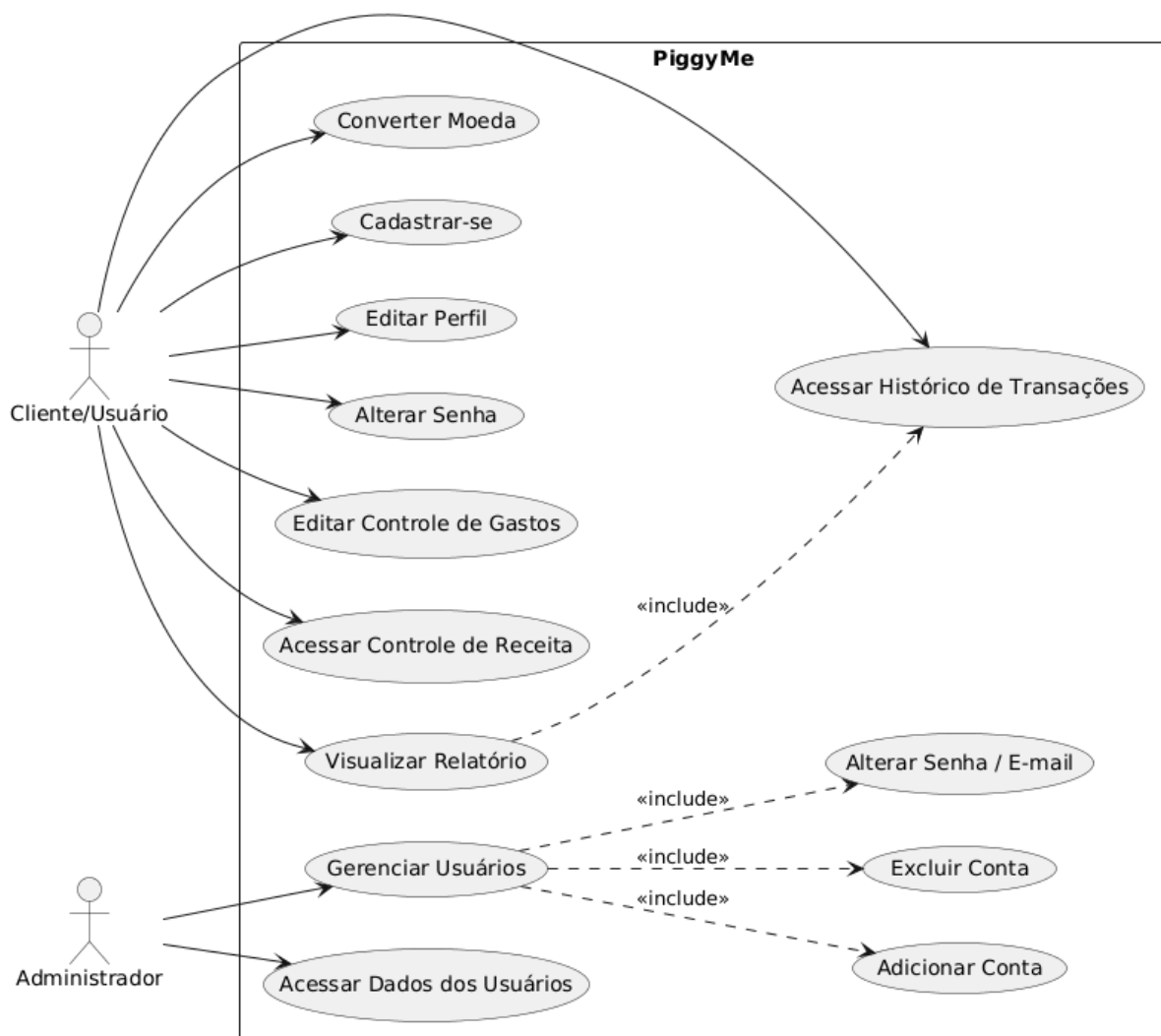
Escopo do Produto

O **PiggyMe** é uma aplicação web responsiva para controle financeiro pessoal. Seu objetivo central é permitir que usuários registrem, categorizem e analisem gastos e receitas, promovendo disciplina financeira por meio de relatórios, conversão de moedas e um sistema gamificado

Diagrama de Casos de Uso

Figura 2 – Diagrama de Casos de Uso do sistema

As interações essenciais entre o usuário e o sistema, abrangendo desde o registro de contas até a gestão de metas financeiras, definem o comportamento esperado da plataforma. O mapeamento desses fluxos e as funcionalidades disponíveis estão descritos no diagrama de casos de uso apresentado na Figura 2

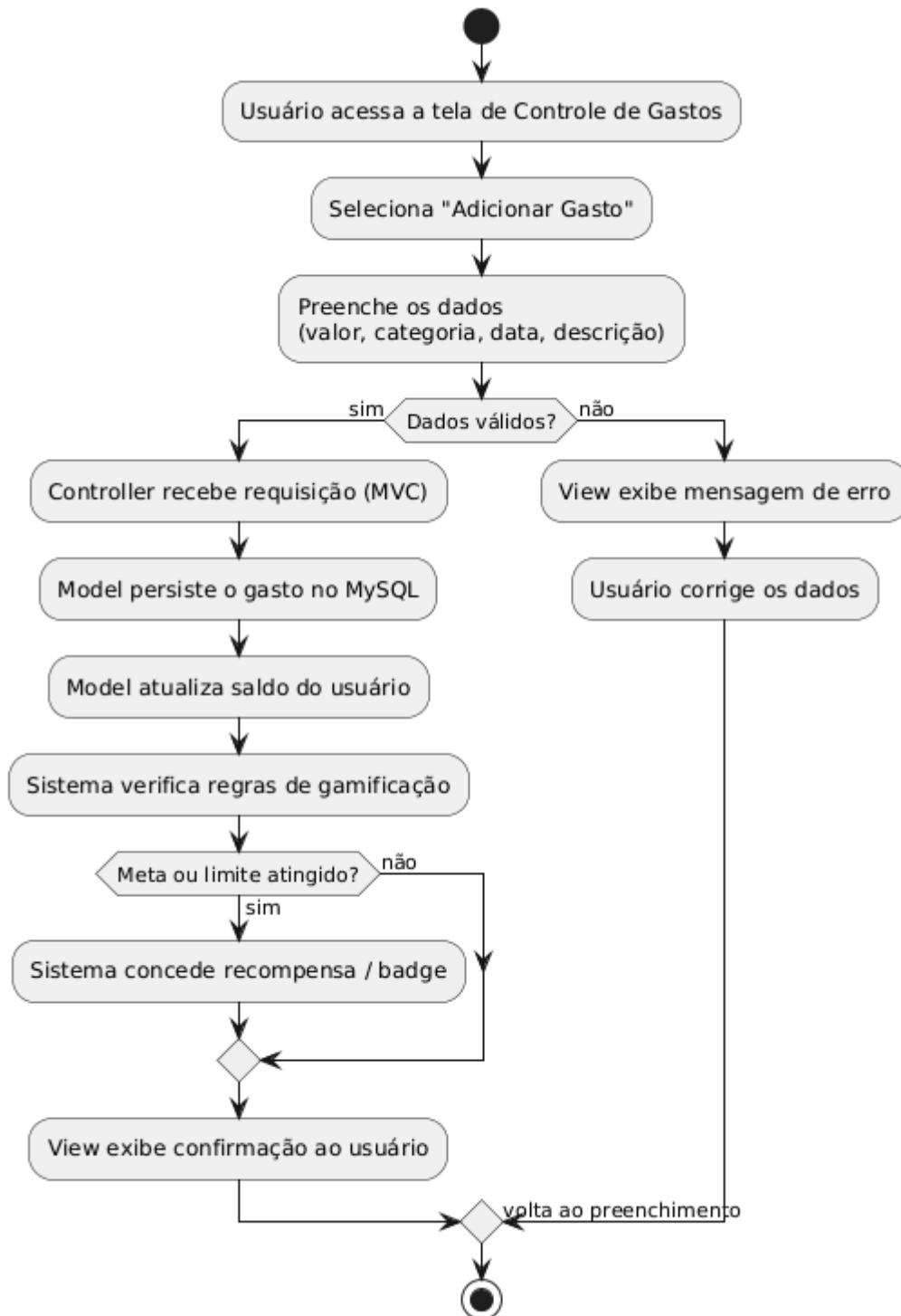


(fonte: autoria própria com o uso plant uml)

Diagrama de Atividades — Controle de Gastos

Figura 3 – Diagrama de Atividades: Controle de Gastos

Para ilustrar o fluxo lógico da funcionalidade principal de controle financeiro, o processo de registro de transações foi modelado para garantir clareza nas etapas. A Figura 3 detalha as atividades de controle de gastos, evidenciando como o sistema processa a entrada de dados financeiros



(fonte: autoria própria com o uso de plant uml)

Requisitos que Influenciaram a Escolha Arquitetural

- A equipe adotou uma arquitetura em camadas no frontend (Apresentação → Lógica de Interface → Comunicação com API) combinada com MVC no backend (Model → View → Controller), pelas seguintes razões:
- Experiência prévia da equipe com a stack JavaScript (front e back) mais MySQL eliminou a curva de aprendizado e viabilizou o desenvolvimento dentro do prazo do projeto.
- O requisito de Agilidade (RF14 - redirecionamentos e relatórios em menos de 1 segundo) favorece uma arquitetura simples e sem overhead de comunicação entre serviços, descartando microsserviços.
- O requisito de Segurança (RF17 - autenticação com tokens/criptografia) mapeia diretamente para uma camada de Controller dedicada a autenticação e middleware no backend, natural no padrão MVC.
- O requisito de Disponibilidade (RF16 - plataforma web/web-mobile) é atendido pela separação clara entre camada de apresentação (frontend responsivo) e camada de negócio (backend independente de plataforma).

2.5.2 Visão de organização lógica

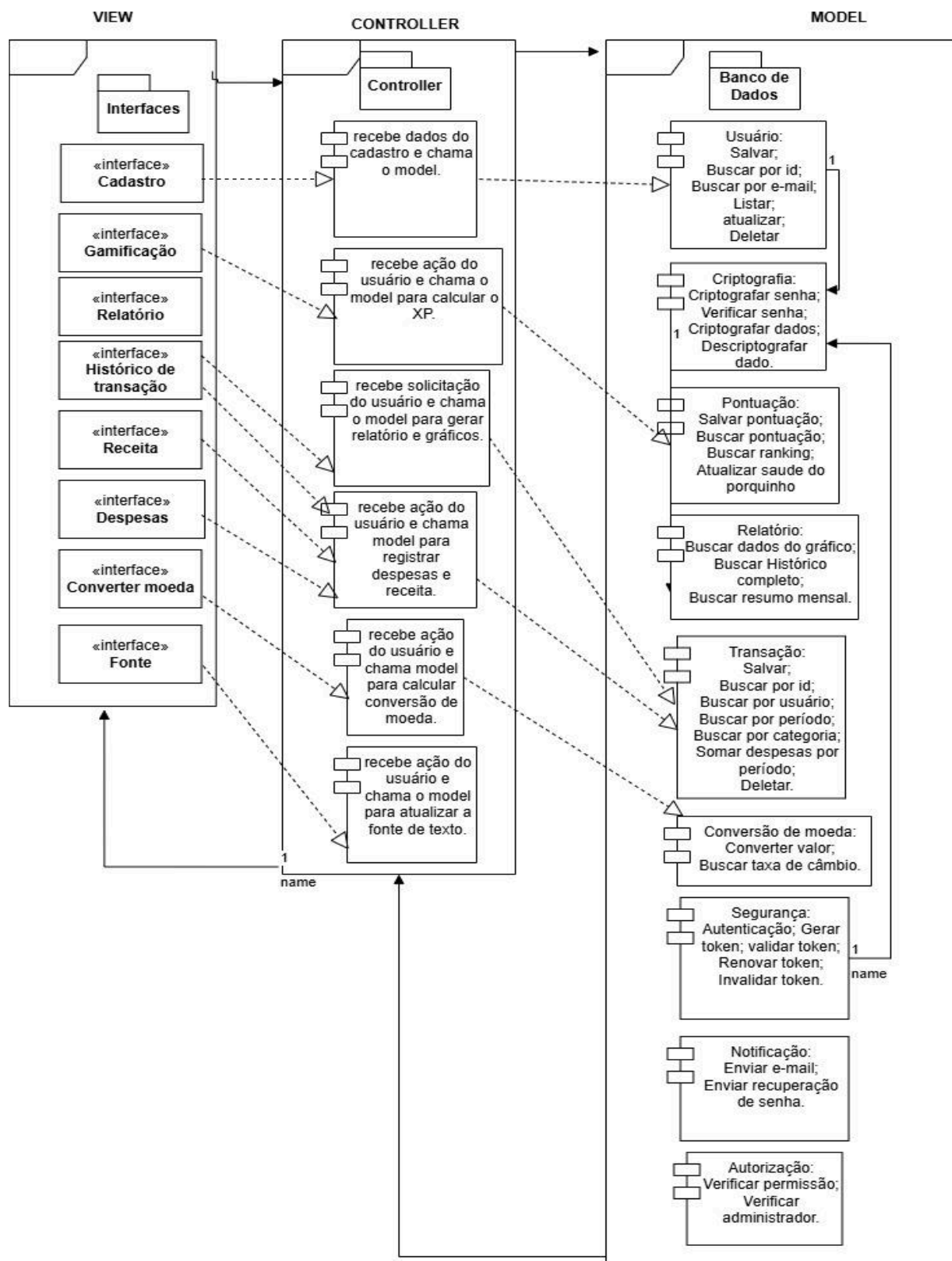
Partindo do princípio de separação de responsabilidades, o software PiggyMe foi dividido em três camadas seguindo o modelo arquitetural Model-View-Controller (MVC). Essa abordagem visa priorizar a manutenção, uniformidade na estrutura e o roteiro de testes do sistema.

O primeiro pacote representa a camada de visualização, View, contendo todos os módulos de interfaces que o usuário pode interagir diretamente, são eles: Cadastro; Gamificação; Relatório; Histórico de transação; Receita; Despesas; Converter moeda e fonte. O segundo pacote representa a camada de controle cuja funcionalidade é receber as solicitações do usuário e se comunicar com a camada Model que realizará a devida operação e retornará o resultado para o controller que sequencialmente atualizará o view. O model está representado pelo terceiro pacote contendo todas as regras de validação e serviços técnicos que compõe os módulos de cadastro de usuário podendo salvar; buscar por id e e-mail; listar; enviar e-mail para redefinir uma senha perdida e até deletar um usuário não mais existente no sistema, cuja responsabilidade é atribuída ao administrador; criptografar e descriptografar as senhas e dados através de configuração de cookies e tokens. Incluem-se ainda os módulos de salvar a pontuação e atualizar a barra de XP referente à solicitação em gamificação, também responsável por executar, validar e retornar todos os dados referentes às transações quanto a despesas, receitas e histórico, buscando por período, por usuário e por categoria, passando por ele também o módulo de conversão de moeda realizando o cálculo e retornando o valor adequado.

Todas as dependências entre os módulos estão representadas por setas com linha tracejada, enquanto as relações e comunicações do sistema estão representadas por setas com linha cheia no Diagrama de pacotes.

Figura 4 – Diagrama de pacotes do sistema

A organização lógica dos módulos do sistema foi definida para promover o baixo acoplamento e a alta coesão entre os componentes. A disposição dos pacotes e suas dependências, conforme apresentada na Figura 4, reflete a divisão de responsabilidades entre as camadas View, Controller e Model.



(Fonte: Autoria Própria.)

2.5.3 Visão Estrutural

A visão estrutural do sistema PiggyMe descreve a organização física e lógica dos componentes que compõem a aplicação, detalhando como esses elementos se conectam e quais são suas responsabilidades específicas. Esta visão é fundamental para compreender a decomposição do sistema em partes menores e gerenciáveis, garantindo que a arquitetura suporte os requisitos de modularidade e manutenibilidade estabelecidos.

Elementos do Sistema e Responsabilidades

O sistema é estruturado em três grandes blocos funcionais, seguindo a stack tecnológica definida (JavaScript, HTML, CSS e MySQL):

1. Frontend (Camada de Apresentação):

- **Responsabilidade:** Prover a interface de usuário responsiva, capturar entradas de dados, validar formulários no lado do cliente e realizar a comunicação assíncrona com o backend.
- **Componentes:** Módulos de autenticação, dashboards de visualização, formulários de registro de transações e componentes de gamificação

2. Backend (Camada de Negócio e Controle):

- **Responsabilidade:** Processar as requisições do frontend, aplicar as regras de negócio (como cálculos de conversão de moeda e lógica de gamificação), gerenciar a autenticação via tokens e realizar operações de CRUD no banco de dados.
- **Componentes:** Controllers (Controle de fluxo), Services (Lógica de negócio), Middlewares (Segurança e validação) e Models (Abstração de dados).

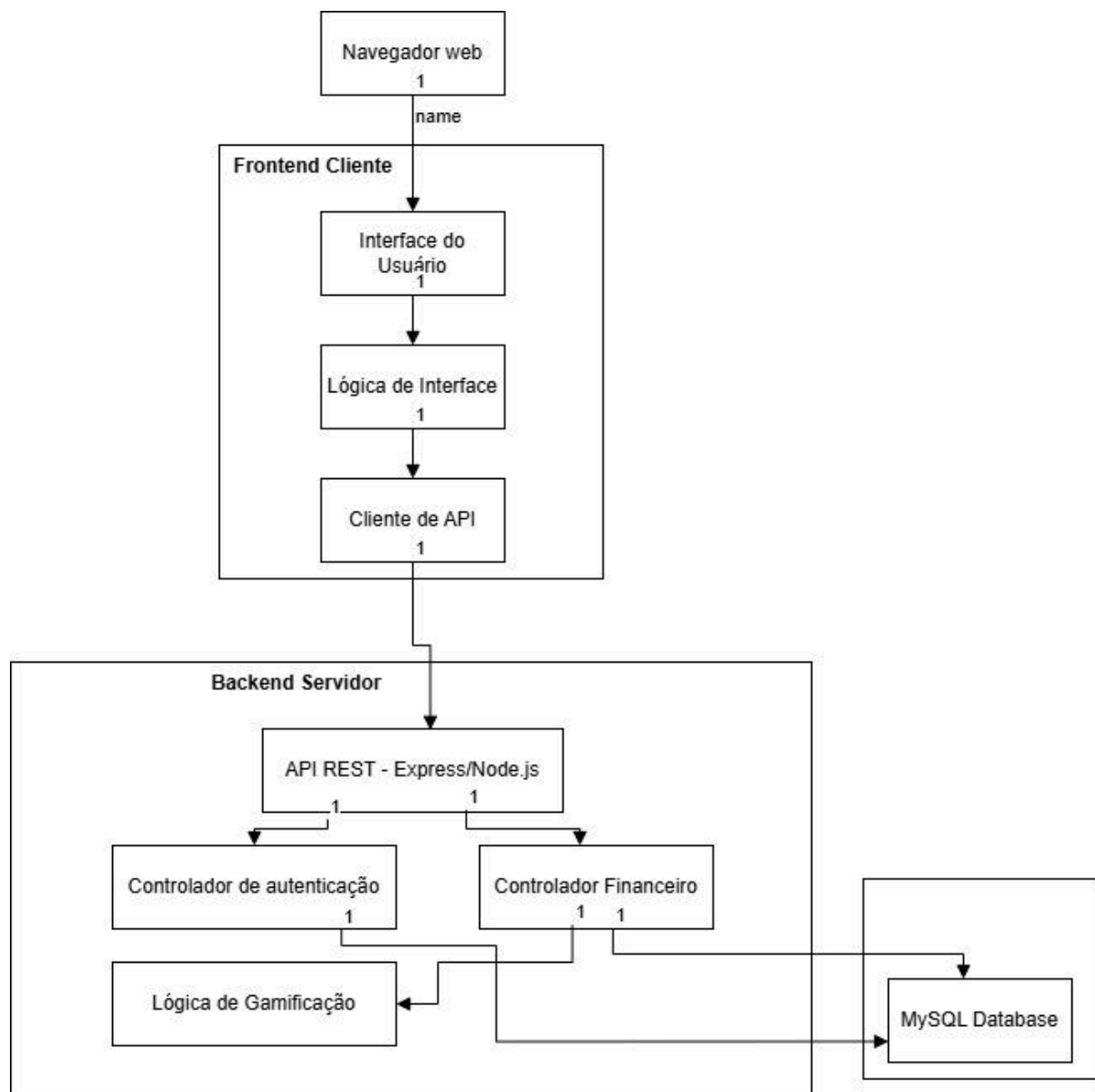
3. Banco de Dados (Camada de Persistência):

- **Responsabilidade:** Armazenar de forma íntegra e segura as informações de usuários, transações financeiras, categorias e metas.
- **Tecnologia:** MySQL.

Diagrama de Componentes

Figura 5 – Diagrama de Componentes

Para garantir a modularidade e permitir que diferentes partes do sistema sejam desenvolvidas de forma independente, o PiggyMe foi decomposto em componentes funcionais. A estrutura desses elementos e suas interdependências técnicas são visualizadas na Figura 5

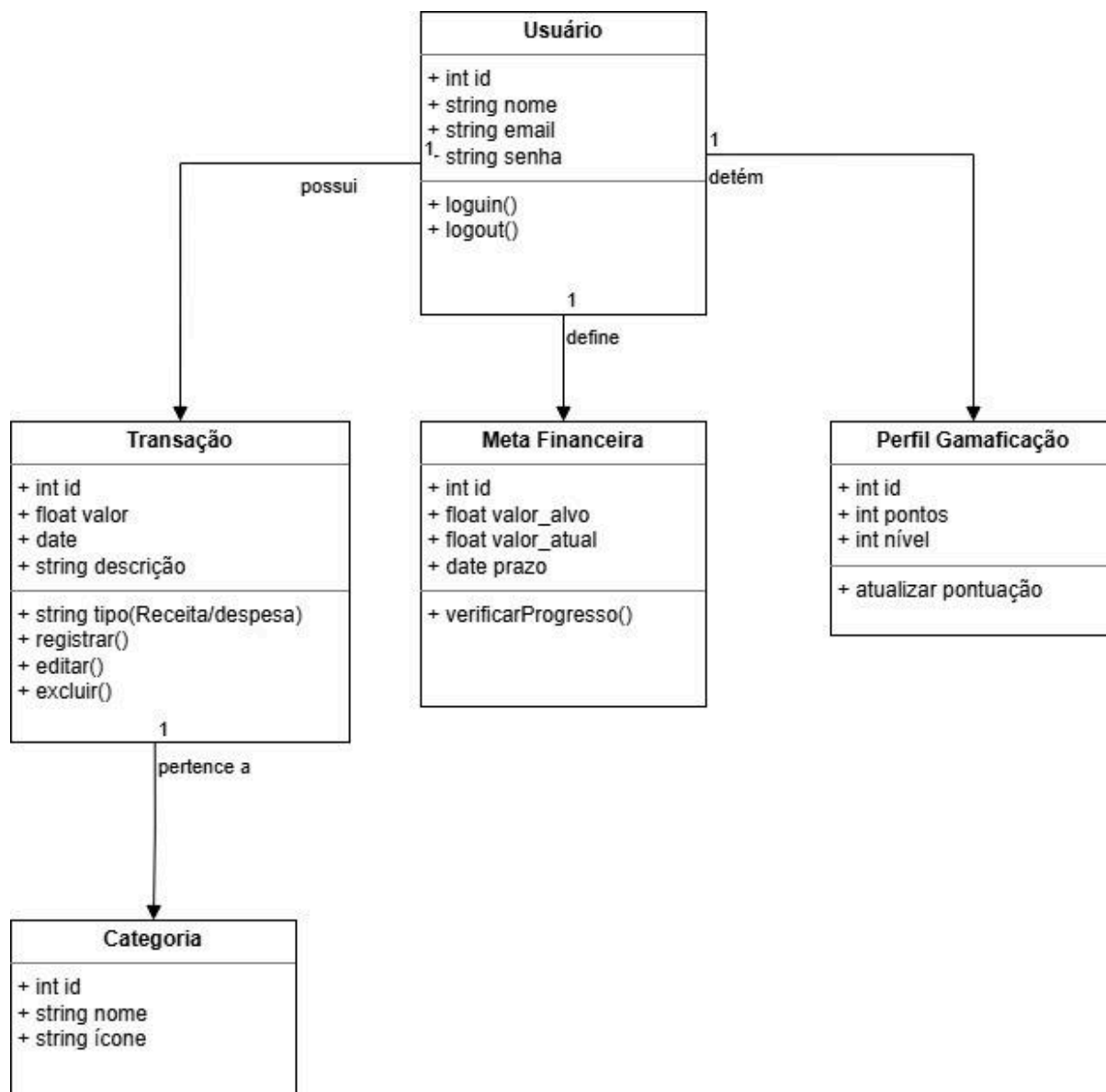


(Fonte: Autoria própria.)

Diagrama de Classes (Modelo de Dados)

Figura 6 – Diagrama de Classes (Modelo de Dados)

O modelo de dados do PiggyMe foi desenhado para persistir informações de forma consistente, garantindo a integridade do histórico do usuário. As principais entidades do sistema e seus relacionamentos, que compõem o modelo de dados, estão representados na Figura 6



(Fonte: Autoria própria.)

Conectores e Protocolos

- **Frontend ↔ Backend**: Utiliza o protocolo HTTP/HTTPS com o padrão REST, onde os dados são trafegados no formato JSON. A autenticação é mantida através de tokens (JWT), garantindo a segurança nas trocas de informações.
- **Backend ↔ Banco de Dados**: A conexão é realizada através de um driver 36 específico para MySQL, utilizando consultas SQL para manipulação dos dados. As

responsabilidades são segregadas para que apenas o backend tenha acesso direto à camada de persistência.

2.6 Visão de Implantação

A visão de implantação descreve a topologia física do sistema, ilustrando como o software é distribuído entre os nós de hardware e como esses nós se comunicam. O PiggyMe utiliza uma arquitetura baseada em nuvem, garantindo acessibilidade via web sem necessidade de instalação local.

Infraestrutura de Hardware e Ambiente de Software

O sistema é distribuído em três nós principais, conectando o usuário final aos serviços de processamento e persistência:

- **Nó do Cliente (User Device):** Representa qualquer dispositivo (Desktop ou Mobile) capaz de executar um navegador web moderno. O ambiente de execução é o próprio navegador, que processa a camada de apresentação (HTML/CSS) e a lógica de interface em JavaScript.
- **Servidor de Aplicação (Backend):** Hospeda a lógica de negócio e os controladores do sistema. Este nó executa um ambiente Node.js, responsável por processar as requisições REST e gerenciar a autenticação via tokens JWT.
- **Servidor de Banco de Dados (Persistence):** Um nó dedicado à execução do MySQL, onde os dados financeiros, usuários e pontuações de gamificação são armazenados de forma íntegra e segura.

Conectividade e Protocolos

A comunicação entre os nós é regida por protocolos de rede que garantem a segurança e a independência das camadas:

1. **Frontend ↔ Backend:** A comunicação ocorre via protocolo HTTPS, utilizando o padrão REST com troca de dados em formato **JSON**. O uso de HTTPS é fundamental para proteger o tráfego dos tokens JWT e dados financeiros sensíveis.
1. **Backend ↔ Banco de Dados:** A conexão é estabelecida por meio de um driver específico para MySQL, utilizando consultas **SQL** em uma rede privada ou protegida por firewall, garantindo que apenas o backend acesse a camada de persistência.

Justificativa da Infraestrutura

A escolha por uma implantação em nuvem (como Heroku, Vercel ou AWS) justifica-se pela meta de disponibilidade e escalabilidade. A separação física entre o servidor de aplicação e o banco de dados reduz o acoplamento e facilita manutenções isoladas, atendendo aos requisitos de manutenibilidade definidos para o projeto.

2.7 Restrições adicionais

O sistema PiggyMe será acessado diretamente pela internet através de navegadores web, sendo necessário que o usuário realize a autenticação por login e senha para acessar suas informações. Essa restrição é necessária para garantir privacidade e segurança dos dados armazenados.

O software será desenvolvido para atender múltiplos usuários simultaneamente, mantendo estabilidade nas principais funcionalidades, como registro de despesas, visualização de gráficos e acesso às metas financeiras.

Características de qualidade

Usabilidade:

O sistema deverá possuir interface intuitiva, simples e responsiva, permitindo fácil utilização por usuários com diferentes níveis de conhecimento tecnológico e financeiro. Essa característica é essencial para incentivar o uso contínuo da plataforma.

Confiabilidade:

As informações financeiras registradas devem permanecer armazenadas corretamente no banco de dados, reduzindo riscos de perda de dados e garantindo consistência das informações apresentadas ao usuário.

Portabilidade:

O PiggyMe deverá funcionar em diferentes dispositivos e navegadores modernos, permitindo acesso tanto em computadores quanto em dispositivos móveis sem necessidade de instalação.

Desempenho:

As funcionalidades principais do sistema deverão apresentar respostas rápidas durante a navegação, cadastro e consulta de informações financeiras, proporcionando melhor experiência ao usuário.

Segurança:

O sistema utilizará autenticação individual por usuário, garantindo que cada pessoa tenha acesso apenas às próprias informações financeiras.

Os perfis de acesso previstos são:

- **Usuário comum:** poderá registrar, editar e visualizar apenas seus próprios gastos, metas e progresso na plataforma.
- **Administrador:** responsável por manutenção, gerenciamento técnico e suporte do sistema, sem alterar informações financeiras pessoais dos usuários sem autorização.

Essas restrições e características foram definidas com o objetivo de garantir segurança, acessibilidade, estabilidade e qualidade no funcionamento do PiggyMe, proporcionando uma experiência mais confiável, intuitiva e eficiente aos usuários. Além disso, essas definições contribuem para a proteção das informações financeiras, para a facilidade de utilização da plataforma em diferentes dispositivos e para a manutenção adequada do sistema ao longo do desenvolvimento e futuras expansões de software.

3 Bibliografia

KRUCHTEN, Philippe. The 4+1 View Model of Architecture. IEEE Software, v. 12, n. 6, p. 42–50, nov. 1995.

MDN WEB DOCS. JavaScript. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>. Acesso em: 14 maio 2026.

MYSQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 14 maio 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education, 2019.

VALENTE, Marco Túlio. Engenharia de Software Moderna. Belo Horizonte: autor, 2020. Disponível em: <https://engsoftmoderna.info/>. Acesso em: 14 maio 2026.